

Critical Security Vulnerability Report — Worksheet Attempt and Student Result Manipulation

Executive Summary

A critical authorization and data-integrity vulnerability exists in the worksheet system at **mathiit.live**.

The primary issue is not the worksheet's minor score-calculation error. The more serious issue is that worksheet attempts and their associated data can be manipulated through the underlying API.

If an attacker obtains a student's **student ID** and has sufficient access to the exposed API, they may be able to manipulate that student's worksheet data rather than being restricted to their own attempts.

The demonstrated capabilities include:

- Re-submitting the same worksheet multiple times.
- Producing different results for subsequent submissions.
- Controlling the number of correct/incorrect answers submitted.
- Supplying arbitrary time spent for individual questions.
- Supplying arbitrary total worksheet completion time.
- Changing worksheet attempt status.
- Potentially modifying another student's attempt when their student ID is known.
- Accessing grading-related functionality directly through the API.
- Retrieving correct-answer information through backend endpoints that should not expose it to an untrusted client.

This represents a **critical assessment-integrity and authorization vulnerability**.

1. Unauthorized Student Data Modification

Severity

Critical — Broken Access Control / IDOR / Data Integrity

Description

The worksheet API appears to rely on identifiers supplied by the client without sufficiently enforcing that the authenticated user is authorized to modify the corresponding student's records.

The code demonstrates requests that operate using values such as:

-
- `worksheet_id`
- `attempt_id`

For example, the attempt workflow explicitly uses a student ID when creating and resolving worksheet attempts.

The code also directly queries and modifies worksheet-attempt records through the Supabase REST API.

Impact

If an attacker obtains another student's ID, the attacker may potentially manipulate that student's academic records.

Depending on the database policies, this could include changing:

- worksheet attempts,
- attempt status,
- submitted responses, • completion time,
- question-level timing,
- answers,
- grading results,
- ranking-related data.

This is significantly more serious than merely being able to automate one's own worksheet.

Expected Security Behavior

A student should only be able to access and modify their own attempts.

The backend should enforce authorization based on the authenticated identity, rather than trusting:

```
student_id
```

provided by the client.

For example:

```
authenticated_user_id
    ↓
server determines associated student
    ↓
server checks attempt.student_id
    ↓
ALLOW only if they match
```

A user should never be able to simply substitute another student's UUID and gain access to their records.

2. Worksheet Re-submission / Attempt Manipulation

Severity

Critical — Assessment Integrity

The same worksheet can be submitted repeatedly, potentially producing different outcomes.

The supplied automation can create or resolve attempts and then submit responses through the backend API.

The workflow subsequently marks an attempt as submitted and invokes the grading process again.

Impact

This can potentially allow a student or attacker to repeatedly manipulate an assessment until a desired result is obtained.

For example:

```
Attempt 1 → 10/15
Attempt 2 → 13/15
Attempt 3 → 15/15
```

If the application treats these attempts as legitimate submissions, the resulting academic record may not represent the student's actual performance.

Recommended Fix

The backend should define a strict attempt lifecycle:

```
created
  ↓
in_progress
  ↓
submitted
  ↓
locked
```

Once an attempt reaches `submitted`, clients should not be able to:

- modify answers,
- modify timing,
- change status,
- resubmit the same attempt, • overwrite the result.

Any legitimate retake should create a **new authorized attempt** according to the application's rules.

3. Client-Controlled Question Timing

Severity

High — Assessment Integrity

The client directly supplies question timing information.

The submitted response payload includes:

- `time_spent_seconds`
- `first_answer_at`
- `last_answer_at`
- `answer_changes`

These values are constructed by the client before being sent to the backend.

The same timing information is also inserted into the direct REST submission payload.

Impact

A malicious client can potentially fabricate the apparent amount of time spent on questions.

For example, a worksheet could appear to have been completed in:

```
Question 1 → 45 seconds
Question 2 → 32 seconds
Question 3 → 18 seconds
...
```

even though those timestamps were generated or supplied by an automated client.

Conversely, the client could potentially submit unusually long or short completion times.

Recommended Fix

Timing information used for assessment integrity should be generated and validated server-side.

At minimum:

- server records attempt start time,
- server records submission time,
- server validates timestamps,
- question timestamps are checked for chronological consistency,

- impossible durations are rejected,
 - clients cannot arbitrarily overwrite historical timing data.
-

4. Client-Controlled Total Worksheet Completion Time

Severity

High

The code calculates a total worksheet completion time locally and sends it to the backend when the attempt is submitted.

This means the reported completion duration can potentially be determined by the submitting client.

Impact

If completion time is used for:

- teacher analytics,
- performance analysis,
- rankings,
- anti-cheating mechanisms, • time limits, then client-controlled timing makes those systems unreliable.

Recommended Fix

Calculate authoritative completion duration using server-side timestamps:

```
completion_time =  
server_submission_timestamp -  
server_attempt_start_timestamp
```

The client should not be trusted to provide the authoritative value.

5. Correct Answers Exposed Through the API

Severity

Critical — Assessment Integrity

The provided code retrieves `correct_answers` directly from worksheet data.

The fallback query requests:

```
questions(id, correct_answer, options)
```

from the worksheet question data.

The grading response is also used to obtain question information and correct answers.

Impact

If an untrusted client can obtain the correct answers before completing the worksheet, the assessment can potentially be solved without actually answering the questions.

This is particularly serious for worksheets that are intended to measure independent student performance.

Recommended Fix

Correct answers should never be returned to an ordinary student client before the assessment is legitimately submitted.

The architecture should instead be:

```
Student submits answer
↓
Server receives answer
↓
Server retrieves correct answer
↓
Server grades answer
↓
Server stores result
↓
Student receives score
```

6. Client-Controlled Submission State

Severity

Critical

The client directly sends a request changing the worksheet attempt status to:

```
submitted
```

The supplied code performs a direct REST PATCH against the worksheet-attempt record.

If the same authorization weakness permits arbitrary attempts to be targeted, this could allow manipulation of another student's attempt state.

Potentially affected states include:

- in_progress
- submitted
- pending
- other application-defined states.

Impact

Changing another student's state could:

- prevent them from continuing,
- make an incomplete worksheet appear submitted,
- cause a worksheet to enter a different workflow,
- interfere with teacher review, • alter the student's academic record.

Recommended Fix

Attempt status must be controlled by server-side business logic.

A student client should not be able to arbitrarily execute:

```
PATCH /worksheet_attempts
status = "submitted"
```

The server should determine whether the transition is valid.

7. Ranking / Academic Record Integrity

The submission workflow triggers grading and student-ranking related operations after the attempt is submitted.

Therefore, manipulation of worksheet attempts could potentially propagate into downstream academic systems.

This makes the vulnerability more significant than a simple UI bug.

Potentially affected systems include:

- worksheet scores,
 - student performance statistics,
 - rankings,
 - teacher dashboards,
 - progress tracking, • academic analytics.
-

8. Important Databases exposed through public api

Severity – Critical

We could access student databases (most important) and etc via public api key, this also gives their entire information such as whatsapp number, student batch id, and most importantly student_id that could be used to alter their entire worksheet history with the previous exploits

This is the code used to retrieve the databases

Python

```
from supabase import create_client, Client
url: str = "https://sthomkdkjwxwbolomovx.supabase.co"
key: str =
"eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpc3MiOiJzdXBhYmFzZSIsInJlZiI6InN0aG9ta2Rrand4d2JvbG9tb3Z4Iiwicm9sZSI6ImFub24iLCJpYXQiOiJE3NjY5ODQ3MTMsImV4cCI6IjA4MjU2MDcxM30.Ymt2XKBdr6vCqjfG68V-xhnjB2nBzZTGxK9SBTd9JIE"
supabase: Client = create_client(url, key)
table_name = "students"
response = supabase.table(table_name).select("*").execute()
data = response.data
print(data)
```

This prints the entire students database

Also these tables were also identified and returned values

alerts, students, batches, worksheets, centers, rate_limits, attendance,
chapters, courses, dpp_marks, exam_slot_marks, email_send_state,
whatsapp_message_log, error_logs, leads, live_sessions, question_media,
question_flags
google_oauth_state, payment_refunds, online_payments, online_tests,
common_tests, student_comments

9. Secondary Bug — Incorrect Score Calculation

Severity

Medium/High — Data Integrity

There is also a separate scoring issue where answering one question incorrectly on a 15-question worksheet can result in a score of **13/15** instead of **14/15**.

This should be fixed, but it is **not the highest-priority issue**.

The security and authorization problems should be addressed first because they can allow intentional manipulation of the underlying academic data.

Overall Risk

Critical Findings

1. Potential unauthorized modification of another student's worksheet data
2. Insufficient authorization enforcement around student/attempt identifiers
3. Potential manipulation of submitted attempts
4. Correct answers exposed through backend-accessible data
5. Client-controlled assessment state

High Findings

1. Client-controlled question timing
2. Client-controlled total completion time
3. Potential repeated submission/attempt manipulation

Secondary Finding

1. Incorrect score calculation
-

Recommended Immediate Remediation

1. Fix authorization first

Every request should verify:

```
authenticated_user
  ↓
authorized student
  ↓
authorized worksheet
  ↓
authorized attempt
```

Never trust a `student_id` supplied by the client.

2. Enable and audit Supabase Row Level Security

Verify that users cannot:

- `SELECT` another student's attempts,
- `INSERT` attempts for another student,
- `UPDATE` another student's attempts,
- `DELETE` another student's attempts,
- modify another student's responses.

3. Protect correct answers

Remove `correct_answer` from student-facing queries.

4. Lock submitted attempts

After submission:

```
submitted → immutable
```

Any legitimate retake should create a new attempt.

5. Move grading server-side

The client should submit answers only.

The server should determine:

- correctness,
- score,
- completion duration, • final status.

6. Make timing authoritative

Use server timestamps instead of trusting:

```
time_spent_seconds  
first_answer_at  
last_answer_at
```

provided by the client.

7. Add rate limiting and CAPTCHA

CAPTCHA can reduce automated abuse, but it should be considered an additional layer rather than the actual authorization mechanism.

8. Rotate exposed credentials

The supplied source contains a hard-coded Supabase credential.
That credential should be rotated/revoked and its usage reviewed.

Suggested Security Classification

Primary: Broken Access Control / IDOR

Secondary: Improper Authorization

Secondary: Client-Side Trust / Parameter Tampering

Secondary: Assessment Data Integrity

Secondary: Sensitive Information Exposure

Overall Severity: Critical

The key issue is not that someone can automate a worksheet. The key issue is that the backend appears to allow the client to control **who the attempt belongs to, what responses it contains, how long it took, and what state it is in**, without sufficiently enforcing that the requesting user is authorized to perform those operations.